35. HTTP Servers and Clients (net/http)

35. HTTP Servers and Clients (net/http)

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `35-http-net`

Learning objectives

- Build handlers with net/http
- Use http.Client with timeouts
- Understand middleware chaining
- Serve JSON APIs safely

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Server

```
mux := http.NewServeMux()
mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    _, _ = w.Write([]byte("ok"))
})
srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 *
time.Second}
log.Fatal(srv.ListenAndServe())
```

Always set timeouts. Infinite hangs are outages.

Client

```
client := &http.Client{Timeout: 10 * time.Second}
resp, err := client.Get(url)
```

Reuse clients; do not create one per request. Close response bodies.

Context

`r.Context()` cancels when the client disconnects — respect it in handlers.

Common mistakes (especially from Python instincts)

- Default client with no timeout
- Ignoring resp.Body.Close()
- Blocking handler work without goroutine limits

Exercises

Work in `exercises/35-http-net/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/35-http-net/` until you have an honest failing attempt.

Self-check

1. Why ReadHeaderTimeout?
2. How reuse http.Client?
3. What cancels r.Context()?

Further reading

- https://pkg.go.dev/net/http